Libreto de podcast - Repaso de la prueba Gorila / Candiech

Podcast de repaso de lo que respondimos en la prueba técnica del 4 nov 2026 (Senior DevOps / Platform Engineer: AWS, EKS, Terraform, GitOps con Argo CD). La idea es escucharlo antes de la entrevista con Gorilla Logic, porque lo más probable es que pregunten escenarios parecidos.

Cómo está armado:

- Dos voces: **ANA** (hace de entrevistadora, pregunta y resume) y **SEBAS** (responde, como lo harías tú en la entrevista).
- Narración en español; la **frase de cierre en inglés** de cada respuesta es la que conviene decir tal cual en la entrevista.
- Cada bloque tiene un momento **"¿Cómo se llama?"**: ANA describe el mecanismo y deja una pausa (`[PAUSA 3 s]`) para que digas el nombre en voz alta antes de escucharlo.
- Duración estimada: 25–30 minutos. Cada bloque funciona como episodio corto si lo quieres partir.
- Para generarlo con voz (ElevenLabs, NotebookLM, Google TTS u otro): las acotaciones entre corchetes `[ ]` no se leen.

Fuente: las respuestas de 01, 02, 03, 04 y 05.

Intro

> **ANA:** Bienvenido a este repaso. Hoy vamos a recorrer, uno por uno, los cinco escenarios de la prueba técnica de DevOps y Platform Engineering: un e-commerce que se pone lento con una campaña, un `terraform plan` peligroso contra producción, un incidente en EKS después de un sync de Argo CD, un canary que sale mal, y una herramienta interna en Python llamada `platformctl`.

> **SEBAS:** Y hay una idea que se repite en todas las respuestas, así que la digo de una vez: **primero estabilizar, después encontrar la causa raíz, y por último prevenir**. Y casi siempre: **no tomar atajos en producción**, porque un atajo convierte un problema pequeño en una caída total.

> **ANA:** Antes de arrancar, recordemos los nombres. EKS es Elastic Kubernetes Service, el Kubernetes administrado de AWS, Amazon Web Services. Argo CD es la herramienta de GitOps: Argo CD mira el repositorio Git y hace que el clúster quede igual a lo que dice Git. Y Terraform es la herramienta de infraestructura como código. Vamos con el bloque uno.

Bloque 1 - E-commerce en AWS con 4 veces el tráfico

ANA: El escenario: un e-commerce tipo Amazon, unos diez mil usuarios y mil compras diarias. Marketing lanza una campaña y el tráfico se multiplica por cuatro. Hay un Load Balancer, dos Availability Zones, subredes públicas y privadas, y Kubernetes en las privadas con tres servicios: web, payment e image, que se hablan por HTTP. Primera pregunta: **el sitio se volvió lento desde la campaña. ¿Qué haces?**

SEBAS: Lo trato como un incidente de producción, porque está afectando plata: sitio lento en plena campaña es igual a carritos abandonados. Abro un canal de incidente, se asigna un Incident Commander, y se le avisa a Marketing, a Soporte y a la gerencia con actualizaciones cada 15 o 30 minutos. Reviso si hubo un despliegue hoy, pero como la lentitud empezó con la campaña, el sospechoso principal es la carga, la capacidad, no el código.

ANA: ¿Y la mitigación inmediata?

SEBAS: Escalar horizontalmente a mano: más réplicas de web, payment e image con `kubectl scale`, y subir el tamaño deseado y el máximo del node group. Le doy prioridad de capacidad a payment, porque el flujo de compra es lo que hay que proteger. Si la base de datos es el cuello de botella, la subo de tamaño temporalmente o le agrego read replicas. Y degrado con gracia: apago funciones no críticas, como recomendaciones o imágenes pesadas.

ANA: Para encontrar la causa raíz, ¿qué método usas?

SEBAS: Voy capa por capa con las **Four Golden Signals**: latencia, tráfico, errores y saturación. En el ALB, Application Load Balancer: tiempo de respuesta de los targets, cantidad de requests, errores 5xx, targets sanos y no sanos. En los nodos: CPU, memoria y red en las dos AZ (Availability Zones), `kubectl top nodes`, y si hay pods en Pending porque el Cluster Autoscaler o Karpenter llegó al máximo de nodos o a la cuota de EC2. En los pods: `kubectl top pods`, `kubectl get hpa` para ver si el HPA, Horizontal Pod Autoscaler, está topado en maxReplicas, CPU throttling, OOMKilled, readiness probes fallando, y latencia por servicio con tracing distribuido: OpenTelemetry, Datadog APM o AWS X-Ray. Y las dependencias: la base de datos, el image service sirviendo imágenes sin CDN, la API externa de pagos y los límites del NAT Gateway.

ANA: ¿Y la solución permanente?

SEBAS: HPA en todos los servicios con mínimos y máximos realistas, Cluster Autoscaler o Karpenter con holgura, requests y limits correctos. Imágenes en S3 servidas por CloudFront. ElastiCache con Redis para consultas frecuentes. Timeouts, retries y circuit breakers. Pruebas de carga antes de cada campaña, con k6, JMeter o Locust, y planeación de capacidad con Marketing. Dashboards de golden signals, SLOs (Service Level Objectives) de latencia, y un postmortem sin culpables, blameless.

ANA: La frase para la entrevista:

SEBAS: *"Stabilize first by scaling out and protecting checkout, then find the bottleneck layer by layer using the golden signals, and finally make sure the system scales automatically and is load-tested before the next campaign."*

ANA: Segunda pregunta: **los usuarios del Lejano Oriente siguen lentos, sobre todo con las imágenes.**

SEBAS: Eso ya no es capacidad, es distancia física. Todo se sirve desde una región en Estados Unidos, y desde Asia cada request paga entre 150 y 250 milisegundos de ida y vuelta. Las imágenes, que son lo más pesado y hay muchas por página, pagan ese costo una y otra vez. Primero mido: RUM, Real User Monitoring, por país, con CloudWatch RUM o Datadog RUM, y pruebas sintéticas desde Asia. El quick win es una CDN, Content Delivery Network: CloudFront con edge locations en Asia y las imágenes en S3 como origen, con headers de Cache-Control y nombres de archivo versionados. Después imágenes más livianas: WebP o AVIF, redimensionar con `srcset` o Lambda@Edge, y lazy loading. Para el contenido dinámico, CloudFront o AWS Global Accelerator, que llevan el tráfico por la red troncal de AWS. Y solo si el negocio lo justifica, una segunda región en Tokio o Singapur con Route 53 latency-based routing y Aurora Global Database.

SEBAS: *"The root cause is physical distance. I'd start with the cheapest, highest-impact fix — a CDN with S3 for images plus image optimization — and only move to multi-region if the Asian market justifies the cost and complexity."*

ANA: Tercera: **el payment service se cae al comprar y arrastra a los demás servicios.** Un efecto dominó.

SEBAS: Ahí hay dos problemas: por qué se cae payment, que es la causa raíz, y por qué arrastra a los demás, que es de arquitectura. El efecto dominó pasa por HTTP síncrono sin timeouts, tormentas de reintentos, recursos compartidos sin límites, y liveness probes que dependen de otros servicios. Para mitigar: rollback si hubo despliegue de payment, apagar el checkout con un feature flag para que el resto del sitio siga funcionando, y escalar payment. Para cortar el dominó: timeouts en todas partes, retries con exponential backoff y jitter, circuit breaker con Istio o Linkerd o con Resilience4j, y bulkheads, o sea compartimentos: requests y limits, node group dedicado, connection pools separados. Y a largo plazo, un flujo de compra asíncrono: la orden queda en PENDING, va a una cola SQS o Kafka, unos workers procesan el pago, y la orden pasa a PAID o FAILED, con DLQ, Dead Letter Queue, e idempotency keys para no cobrar dos veces.

SEBAS: *"Services must fail independently: timeouts, circuit breakers and bulkheads, and payments moved to an asynchronous, queue-based flow with idempotency, so a payment failure never takes down the whole website again."*

¿Cómo se llama? (Bloque 1)

ANA: Te describo el mecanismo y tú dices el nombre. Un patrón que, cuando un servicio falla muchas veces seguidas, deja de llamarlo por un rato y responde de una vez con error, para no seguir esperando ni empeorar la carga. ¿Cómo se llama? **[PAUSA 3 s]**

SEBAS: **Circuit breaker.**

ANA: Separar recursos, como pools de conexiones o node groups, para que si una parte se inunda, las otras sigan a flote, como los compartimentos de un barco. **[PAUSA 3 s]**

SEBAS: **Bulkhead.**

ANA: Un valor único por operación de pago, para que si la misma petición llega dos veces, no se cobre dos veces. **[PAUSA 3 s]**

SEBAS: **Idempotency key.**

ANA: El patrón de transacciones distribuidas donde cada paso tiene una acción de compensación si algo falla más adelante. **[PAUSA 3 s]**

SEBAS: **Saga pattern.**

Bloque 2 - SaaS en EKS con Terraform

ANA: Ahora estás de on-call en un SaaS sobre AWS. EKS, VPC (Virtual Private Cloud), subredes, security groups, IAM (Identity and Access Management) y node groups, todo en Terraform con state remoto compartido. Los applies los hace CI después de revisión. Primera pregunta: **te llega un `terraform plan` contra producción: 6 para agregar, 4 para cambiar, 2 para destruir. ¿Lo apruebas, lo bloqueas o pides que lo partan?**

SEBAS: **Split / rework.** No lo aplico a producción así como está. Tres de los cuatro cambios —el IAM role, la subnet y la security group rule— son refactors de código que Terraform interpreta como destruir y crear, aunque la infraestructura real no debería cambiar. Esos tienen que quedar en cero cambios usando **`moved` blocks** o `terraform state mv`. El único cambio real es el launch template del node group de GPU, de la versión 12 a la 13, y ese va en su propio PR. Además señalo una inconsistencia: el resumen dice 2 destroy y el extracto muestra 3. No apruebo un plan de producción que no puedo ver completo; pido el plan guardado de CI con `terraform show tfplan`.

ANA: ¿Cuál es el ítem de más riesgo?

SEBAS: El IAM role de los nodos, `-/+ aws_iam_role.eks_nodes`, "forces replacement, address changed". Como los nombres de IAM role son únicos, primero se destruye el viejo y luego se crea el nuevo. En esa ventana, todos los nodos de EKS pierden su identidad: el kubelet no se autentica y los nodos quedan NotReady, la VPC CNI no puede asignar IPs a los pods, y no se pueden bajar imágenes de ECR, Elastic Container Registry. Es riesgo de caída de todo el clúster. Y aunque el nombre sea igual, el role nuevo tiene otro role ID, así que las políticas de KMS, de S3 o de confianza cross-account que referenciaban al viejo dejan de funcionar en silencio. Pido el `moved` block y le agrego `lifecycle { prevent_destroy = true }`.

ANA: ¿Y la subnet?

SEBAS: La llave del `for_each` cambió de nombre de AZ, `us-east-1b`, a ID de AZ, `use1-az2`. Terraform ve dos recursos distintos y quiere destruir una subred privada de producción. Lo más probable es que AWS rechace el borrado con DependencyViolation y el apply quede a medias. Y ojo: los nombres de AZ se mapean distinto por cuenta, así que `us-east-1b` no necesariamente es `use1-az2`. Pido la prueba con `aws ec2 describe-availability-zones`. Si coinciden, `moved` block. Si no coinciden, eso ya no es un refactor sino una migración de red con su propio diseño.

ANA: ¿Y cómo lo previenes?

SEBAS: Policy-as-code en CI, con OPA/Conftest, Checkov o Sentinel, que bloquee planes de producción con destrucción o reemplazo de IAM roles, subredes o node groups sin un override explícito.

ANA: Segunda pregunta: **un apply de CI se quedó en timeout y dejó el state lock tomado. ¿Qué haces antes de un `force-unlock`?**

SEBAS: El principio es: **el lock no es el problema, el lock nos está protegiendo.** El riesgo real es un state que ya no coincide con AWS. Son siete pasos:

1. **Congelar** los cambios en producción y pausar el pipeline, para que ningún job en cola agarre el lock apenas se libere.
2. **Verificar si el dueño del lock de verdad murió.** Que haya dejado de escribir logs no significa que el proceso paró. Si sigue vivo, se espera o se detiene con **SIGINT**, para que Terraform termine, escriba el state y suelte el lock solo.
3. **Respaldar el state** con `terraform state pull`, revisar versiones del objeto en S3, y buscar el **`errored.tfstate`** en el runner: Terraform lo escribe localmente cuando no puede subir el state, y puede ser el único registro de lo que se creó.
4. **Saber qué cambió en AWS** con el plan guardado y con **CloudTrail**.
5. Solo si todo eso se cumple: **`terraform force-unlock`** con el Lock ID exacto.
6. **Reconciliar** con `terraform plan -refresh-only`, y traer recursos huérfanos con **`import` blocks**, en vez de volver a aplicar y duplicarlos.
7. **Retomar por CI** y hacer el postmortem.

SEBAS: Y el cambio para que no se repita: que CI detenga a Terraform con SIGINT y un periodo de gracia, en vez de matarlo, y con un timeout por encima del apply más largo esperado.

ANA: Tercera: **drift. Durante un incidente alguien agregó a mano el CIDR `10.55.8.0/24` a un security group, y un compañero propone poner `lifecycle ignore_changes` para que Terraform deje de quejarse.**

SEBAS: Rechazo el `ignore_changes` amplio: deja ese security group como un punto ciego permanente, donde Terraform aceptaría en silencio hasta un `0.0.0.0/0`. **El drift es una señal; el objetivo es que Terraform vuelva a ser la fuente de verdad, no silenciar la señal.** Pero tampoco corro un apply normal ahora, porque Terraform borraría la regla y volvería la caída. Verifico en CloudTrail qué se abrió exactamente, qué es ese rango con route tables e IPAM, y con **VPC Flow Logs** si el tráfico es real. Después codifico la regla acordada en Terraform con el ticket del incidente en un comentario, hasta que el plan salga limpio. Si la regla final es más estrecha, la aplico como **"add before remove"**. Y de salvaguarda: **drift detection** programada con `terraform plan -detailed-exitcode`, alertas de AWS Config o EventBridge para cambios fuera del rol de CI, y un break-glass que obligue a codificar el cambio en 24 o 48 horas.

ANA: Cuarta, en video de 60 segundos: **un stakeholder te pide `apply -auto-approve`, `force-unlock` y `-target` hoy mismo.**

SEBAS: *"I understand this is urgent, and I want to get it out today. But I won't take those shortcuts, because each one can turn a small change into a production outage. Auto-approve means applying something nobody has reviewed. Force-unlocking a running apply can corrupt our state. And `-target` leaves the state half-applied, which usually breaks the next deployment. Here's what I propose: I'll push the change through our normal pipeline right now, review the plan with a teammate within the next thirty minutes, and if it only touches the node group as expected, we apply it today through CI, while I monitor the service. If the plan shows anything unexpected, I'll tell you right away with the specific risk and a new ETA. That way we ship today, without putting customers at risk."*

ANA: Fíjate en la estructura, que sirve para cualquier stakeholder: **entiendo la urgencia → no al atajo y por qué → mi alternativa con tiempos → qué pasa si hay sorpresas.**

¿Cómo se llama? (Bloque 2)

ANA: Le dices a Terraform que un recurso solo cambió de dirección en el código, para que no lo destruya y lo vuelva a crear. Se escribe en el código, no por línea de comandos. **[PAUSA 3 s]**

SEBAS: **El `moved` block.** Y la versión por comando es `terraform state mv`.

ANA: El archivo que Terraform escribe en el disco local cuando no logra subir el state al backend. **[PAUSA 3 s]**

SEBAS: **`errored.tfstate`.**

ANA: El comando que muestra la diferencia entre el state y la realidad sin proponer ningún cambio de infraestructura. **[PAUSA 3 s]**

SEBAS: **`terraform plan -refresh-only`.**

ANA: La señal que se le manda a Terraform para que pare con calma, termine lo que estaba haciendo y libere el lock. **[PAUSA 3 s]**

SEBAS: **SIGINT.**

ANA: La práctica de revisar los planes con reglas automáticas en CI, con herramientas como OPA o Checkov. **[PAUSA 3 s]**

SEBAS: **Policy-as-code.**

Bloque 3 - Incidente en EKS después de un sync de Argo CD

ANA: SaaS B2B multi-tenant en EKS. A las 14:01 hay un sync de Argo CD; desde las 14:03, errores 5xx. El HPA pide réplicas pero los pods nuevos no arrancan con `FailedCreatePodSandBox ... failed to assign an IP address`. IPs libres por subred: a tiene 3, b tiene 1, c tiene 900. El ALB devuelve 503 con 0 targets sanos y 6 no sanos. Y el Ingress tiene `target-type: instance` con un Service de tipo `ClusterIP`. **Escribe la nota de incidente en máximo cinco bullets.**

SEBAS: La clave es ver que son **dos problemas**. El 503 total lo explica `target-type: instance` con un Service `ClusterIP`: en modo instance, el ALB le manda el tráfico a los nodos por un **NodePort**, y un Service `ClusterIP` no tiene NodePort, entonces todos los targets fallan el health check. Eso coincide con el sync de las 14:01. La **IP exhaustion** de la **VPC CNI** en las subredes a y b es real, pero secundaria: impide escalar, pero no explica que los pods viejos estén no sanos.

ANA: Los cinco bullets.

SEBAS: Uno, **impacto**: caída total de la API pública desde las 14:03 para todos los tenants, dos minutos después del sync. Dos, **contención**: rollback del release por GitOps, con `git revert` o rollback de Argo CD con el auto-sync pausado, sin ediciones manuales con `kubectl` que Argo CD va a sobrescribir. Tres, **chequeo de Kubernetes**: el diff de

Service e Ingress entre el sync y la revisión anterior, y los logs del AWS Load Balancer Controller. Cuatro, **chequeo de AWS**: `aws elbv2 describe-target-health` para ver los reason codes y comparar el puerto registrado con el NodePort. Cinco, **hipótesis principal**: el release cambió el Service de NodePort a ClusterIP, o la anotación de `ip` a `instance`; al revertir, los targets deberían quedar sanos en unos pocos intervalos de health check.

ANA: Segunda, video de 30 segundos: **un compañero propone hacer cambios en vivo fuera de GitOps y "reconciliar Git y Terraform después".**

SEBAS: *"I agree, getting traffic back is the priority. But live edits won't stick: Argo CD will revert them, EKS add-ons get overwritten, and we lose track of what actually fixed or broke things. We already have a strong suspect, the 14:01 release, so the fastest safe fix is reverting it in Git and letting Argo CD sync. If that fails, we do one scoped break-glass change, logged here, with auto-sync paused, and codified in Git right after, not 'later'."*

ANA: Tercera, 30 segundos: **un PM pregunta si hacer rollback o seguir investigando.**

SEBAS: *"We roll back now, and keep investigating in parallel. The errors started two minutes after the release, it's a full outage for every customer, and rolling back returns us to a version we know works. I'll do a quick check that the release has nothing irreversible, like database migrations. We've already saved the evidence, so we'll still find the root cause. I'll update you in fifteen minutes."*

¿Cómo se llama? (Bloque 3)

ANA: El componente de EKS que le da a cada pod una IP real de la subred de la VPC, y que por eso puede dejar sin IPs a una subred. **[PAUSA 3 s]**

SEBAS: **La VPC CNI**, Container Network Interface. El problema se llama **IP exhaustion**.

ANA: El tipo de Service que necesita el ALB cuando el Ingress usa `target-type: instance`. **[PAUSA 3 s]**

SEBAS: **NodePort.**

ANA: La función de Argo CD que devuelve automáticamente al estado de Git cualquier cambio manual hecho en el clúster. **[PAUSA 3 s]**

SEBAS: **Self-heal**, junto con el auto-sync de Argo CD.

Bloque 4 - Canary con progressive delivery en GitOps

ANA: Release on-call en un SaaS B2B. A las 10:12 se mergea el PR de promoción; a las 10:14 el sync manda el 10% al candidato; 10:17, el canary tiene más 5xx; 10:18, la latencia empeora en todo el servicio, **incluido el stable**; 10:19, alerta en una dependencia downstream compartida. La imagen se referencia por **tag**, no por digest, y ese tag se empujó varias veces en la última hora. Argo CD con auto-sync. **¿Tus cuatro acciones en los próximos diez minutos?**

SEBAS: Uno: **abortar el canary** con el controlador, `kubectl argo rollouts abort`, y el 100% vuelve al stable. Lo hago con el mecanismo del controlador porque cambia el status del rollout y no el spec, así el auto-sync de Argo CD no pelea con eso. Dos: **mirar las métricas y traces del servicio y de la dependencia** y comparar las llamadas del candidato contra las del stable, para saber si el candidato causó una tormenta de reintentos o si la dependencia tiene su propio problema. Tres: **verificar qué imagen corre en cada cohorte** por **digest**, con el `imageID` de los pods, comparado con el historial del tag en ECR. Como el tag se empujó varias veces, es posible que los pods "stable" hayan bajado la imagen nueva con el mismo tag al reiniciar, y eso explicaría por qué el stable también se degradó. Cuatro: **hacer que Git coincida con la decisión**: revertir el PR de las 10:12 y fijar la última versión buena **por digest**, `image@sha256`. Seguimiento: activar **ECR tag immutability**.

ANA: Segunda, 30 segundos: **un stakeholder pregunta si no se puede hacer un hot-fix directo en producción y limpiar Git después.**

SEBAS: *"Good news: the errors are already contained. We stopped the new version and all traffic is back on the stable one. A direct hot-fix wouldn't even last: our platform automatically undoes manual changes within minutes, so the errors could come back. Making the fix through Git takes the same few minutes, and it actually sticks. I'm doing that now, and I'll update you in ten minutes."*

ANA: Tercera, 30 segundos: **el rollout falló, pero incluía una migración de base de datos. ¿Rollback o roll-forward?**

SEBAS: *"First, let's pause the rollback: the old version may not work with the new schema, and we could cause a second outage. I need three answers: was the migration additive or destructive, did it fully complete, and has new data already been written in the new format? If it's backward-compatible, we roll back the app. If not, we roll forward with a fix, because undoing the migration could lose data. Either way, I confirm we have a recent database snapshot first."*

ANA: Y si te preguntan cómo evitarlo para siempre…

SEBAS: El patrón **expand-contract**, también llamado **parallel change**: primero solo se agregan columnas o tablas, después se migra el código, y en un release posterior se borra lo viejo. Así el rollback de la app siempre es seguro.

¿Cómo se llama? (Bloque 4)

ANA: La referencia inmutable a una imagen de contenedor, basada en el hash de su contenido. **[PAUSA 3 s]**

SEBAS: **El image digest**, el `sha256`.

ANA: La configuración del registro de AWS que impide volver a empujar una imagen con un tag que ya existe. **[PAUSA 3 s]**

SEBAS: **ECR tag immutability.**

ANA: El controlador de Kubernetes que hace canary y blue-green y se integra con Argo CD. **[PAUSA 3 s]**

SEBAS: **Argo Rollouts.** La alternativa es **Flagger**.

> **ANA:** Migrar el esquema en tres pasos —agregar, migrar el código, borrar lo viejo— para que el rollback siempre sea seguro. **[PAUSA 3 s]**

> **SEBAS:** **Expand-contract**, o **parallel change**.

Bloque 5 - La herramienta interna `platformctl` en Python

> **ANA:** El último. Un script en Python que el on-call corre desde el portátil y a veces desde CI, para inspeccionar workloads y reiniciar pods. Usa `subprocess.run` con `shell=True`, busca el clúster con el primer nombre que contenga `env` sin paginar, hace `log.info` de todo `os.environ`, filtra con `grep CrashLoopBackOff`, borra pods uno por uno, tiene `ENV=prod` por defecto y al final siempre imprime `done`. **Dame cuatro problemas.**

> **SEBAS:** Uno: **filtra secretos a los logs.** `dict(os.environ)` incluye `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN` y tokens de CI. Una herramienta para reducir riesgo se vuelve un vector de fuga de credenciales. Nunca se loguea el entorno; solo campos explícitos.

Dos: **puede apuntar al clúster equivocado, o a producción por defecto.** "prod" también coincide con "preprod" o "nonprod", no pagina con `nextToken`, devuelve `None` sin fallar, y `use-context` cambia el kubeconfig global del ingeniero. La mejora: `ENV` obligatorio, nombre exacto con paginación, fallar si hay cero o más de un resultado, y pasar `--context` en cada llamada.

Tres: **command injection y parsing frágil.** Con `shell=True` y f-strings, un selector como `app=x; kubectl delete ns payments` ejecuta comandos arbitrarios. Y el `grep` sobre texto puede coincidir con otras columnas. La mejora: lista de argumentos con `shell=False`, o el cliente oficial de Kubernetes en Python, y salida estructurada con `-o json`.

Cuatro: **falla en silencio y sin límites.** No revisa return codes, no tiene timeouts, siempre dice `done`, el selector vacío puede borrar todo, y no hay dry-run. En un incidente, el ingeniero ve `done` y cree que mitigó cuando todo falló. La mejora: fail fast con exit code distinto de cero, dry-run por defecto, máximo de pods por corrida, y logging estructurado.

> **ANA:** Segunda: **diseña `platformctl v2`.**

> **SEBAS:** Dos subcomandos con inputs obligatorios: `platformctl rollout status`, de solo lectura, y `platformctl pods restart`, que muta, con `--env`, `--namespace`, `--deployment` y `--reason` obligatorios. El `--env` se mapea por un archivo de configuración versionado a un clúster, cuenta y región exactos, y se verifica con `sts get-caller-identity`. **Dry-run por defecto**: sin `--execute` solo muestra el plan, y en producción hay que escribir el nombre del ambiente para confirmar. Los reinicios van por la **Eviction API**, respetando los **PodDisruptionBudgets**, en lotes pequeños, esperando que los reemplazos queden Ready, y sin tocar el spec del Deployment para no crear drift con Argo CD. Para un humano: una línea final clara de SUCCESS o FAILED, con resultados en **stdout** y diagnóstico en **stderr**. Para un sistema: **exit codes documentados** —0 éxito, 1 falló, 2 input inválido, 3 rechazado por guardrail, 4 éxito parcial, 5 timeout o autenticación— y `--output json` con un esquema versionado. Y auditoría: quién corrió qué, contra qué clúster y por qué incidente.

ANA: Tercera, video de 60 segundos: **un product engineer te pide correr el restart amplio en todo producción.**

SEBAS: *"I get it, we all want this fixed as fast as possible, and I'm with you on that. But a broad restart would also restart everything that's healthy. We'd lose capacity all at once, every service would hit the database at the same time, and we could turn a partial problem into a full outage. We'd also wipe the logs we need, and if the cause is somewhere else, things just get stuck again. Here's what I suggest: give me two minutes to check which services are actually stuck with our status command. Then I restart only those, in small batches, watching that they come back healthy. It's about the same time, and much safer. You can really help here: tell me which customer flows are failing, so we start with those. I'll keep you updated in the channel."*

¿Cómo se llama? (Bloque 5)

ANA: La API de Kubernetes que saca un pod respetando los límites de disponibilidad, en vez de borrarlo a la fuerza. **[PAUSA 3 s]**

SEBAS: **La Eviction API.**

ANA: El objeto de Kubernetes que dice cuántos pods de una aplicación pueden estar caídos a la vez durante una interrupción voluntaria. **[PAUSA 3 s]**

SEBAS: **PodDisruptionBudget**, PDB.

ANA: El comando de AWS que te dice con qué identidad y en qué cuenta estás autenticado. **[PAUSA 3 s]**

SEBAS: `aws sts get-caller-identity`.

Cierre - Las cinco ideas que hay que llevarse a Gorilla Logic

ANA: Para cerrar, las ideas que atraviesan todo:

SEBAS: Uno: **estabilizar, encontrar la causa, prevenir**, en ese orden.

ANA: Dos: **Git es la fuente de verdad.** Los cambios manuales no duran porque Argo CD los revierte; el arreglo más rápido y seguro es revertir en Git. Y si hace falta un break-glass, que sea uno solo, registrado, con auto-sync pausado y codificado de inmediato.

SEBAS: Tres: **en Terraform, el lock y el drift son señales, no estorbos.** Nada de `-auto-approve`, `force-unlock` a ciegas, `-target` o `ignore_changes` amplio.

ANA: Cuatro: **los servicios tienen que fallar de forma independiente**: timeouts, circuit breakers, bulkheads, colas e idempotencia.

SEBAS: Y cinco: **con un stakeholder, la fórmula es**: entiendo la urgencia, este atajo es riesgoso por esto, mi alternativa toma casi el mismo tiempo, y te actualizo en tantos minutos.

ANA: Último repaso rápido de nombres, sin pausa, para que te queden sonando: Four Golden Signals, HPA, Karpenter, CloudFront, circuit breaker, bulkhead, idempotency key, Saga, `moved` block, `prevent_destroy`, `errored.tfstate`, SIGINT, `plan -refresh-only`, `import`

block, policy-as-code, VPC CNI, NodePort, self-heal, Argo Rollouts, image digest, ECR tag immutability, expand-contract, Eviction API y PodDisruptionBudget.

SEBAS: Mucha suerte en la entrevista.